

Category 1: Configuration, Mocks & Error Handling

1. Config.js

- **What it does:** Acts as the central settings dashboard for your agent.
- **How it works:** Instead of hardcoding API keys or email addresses, this file reads settings from Google Apps Script's **Script Properties** (like environment variables). It has fallback defaults (e.g., standard timezones, default spreadsheet mappings) and allows the testing suite to inject temporary settings in memory.
- **Key settings managed:** ZeroBounce API keys, EmailOctopus API keys/secrets, Calendar IDs, Rachel's email, and sending modes (`auto` or `manual`).

2. Services.js

- **What it does:** Creates a mockable wrapper around Google Apps Script's built-in services.
- **How it works:** GAS provides global objects like `UrlFetchApp` (for API requests), `MailApp` (for sending emails), and `DriveApp` (for file management) that only work when deployed inside Google's cloud. This file wraps them inside simple JS objects (`Http`, `Mail`, `Drive`, etc.). During tests, we can override these objects in memory to simulate sending emails, scanning folders, or getting API errors without making real external calls.

3. ErrorHandler.js

- **What it does:** Makes sure the agent fails loudly, safely retries failed tasks, and alerts Rachel when things break.
- **How it works:**
 - It logs errors to the `Error Log` sheet.
 - It wraps API calls with an **exponential backoff policy** (tries a failed task up to 3 times, waiting 2 seconds, then 4 seconds, then 8 seconds before placing it in a "dead-letter" queue).
 - If a critical error occurs (like the database or EmailOctopus being unreachable), it sends Rachel an immediate "Pipeline Down" alert.
 - Otherwise, it batches minor errors into a clean, daily HTML digest. It also daily monitors API credit levels (like ZeroBounce credit drops).

Category 2: Data Validation & Merging

4. Schema.js

- **What it does:** Sanitizes, normalizes, and validates all contact data before it is saved.
- **How it works:** It forces emails to lowercase, cleans them, and calculates an MD5 hash (the subscriber's unique ID). It formats names into proper title casing (while keeping internal capitals like `McLeod` or `O'Brien`), normalizes phone numbers to standard E.164

formats, sorts/semicolon-joins tags, and sanitizes input text to prevent formula injection in Google Sheets (e.g., prepending a ' to values starting with =, +, -, or @).

5. Database.js

- **What it does:** Manages direct CRUD (Create, Read, Update, Delete) operations on your Google Sheet.
- **How it works:** It checks if tabs like `Master Contacts`, `Error Log`, `Audit Log`, and `Email Activity` exist (creating them with standard headers if missing). It maps rows to JavaScript objects and handles scanning rows to lookup existing users, appending new ones, or writing entries to the audit and email activity logs.

6. DedupeMerge.js

- **What it does:** Merges new sign-up details with existing contact profiles and handles state transitions.
- **How it works:**
 - It wraps every look-up-and-write in a **Script Lock** (`LockService`) to prevent race conditions during rapid signup bursts (Scenario 1).
 - If a contact exists, it merges data field-by-field: empty fields are populated, stable fields are preserved, and conflicts in mutable fields are overwritten with the new value while the old value is recorded in the `Audit Log` (Scenario 21).
 - It validates lifecycle state transitions (e.g., making sure a contact cannot teleport from `new` to `won` without booking first).
 - For LinkedIn manual imports, it enforces a rule that only adds tags/empty fields and never overwrites populated ones.

Category 3: Source Ingestion Connectors

7. GoogleForms.js

- **What it does:** Listens to Google Form submissions and syncs them.
- **How it works:** Triggered instantly on form submit. It maps questions case-insensitively to schema fields (like `Email Address` to `email`). If a question cannot be mapped, it alerts Rachel immediately (Scenario 22) so she knows someone edited the form and broke the layout. It verifies the email, merges/writes the data to the sheet under lock, and syncs the subscriber to EmailOctopus.

8. LumaPoll.js

- **What it does:** Periodically processes Luma workshop registration exports.
- **How it works:** Runs every 15 minutes. It scans a specific Google Drive folder (`Arkay/Luma-Inbox`) for CSV files. It processes them one by one (never concurrently, preventing duplicates - Scenario 23). To survive Google's 6-minute script execution limit, it saves a cursor (current file ID and row index) so it can resume exactly where it was cut off (Scenario 9). Once a CSV is fully processed, it moves the file to `Arkay/Luma-Done`.

9. LinkedInImport.js

- **What it does:** Allows administrators to manually import CSV lists of LinkedIn connections.
- **How it works:** Cleans the file by stripping emojis and control characters, applies the LinkedIn merge rules (only populates empty columns and appends tags, never overwriting existing master contact data), verifies emails via ZeroBounce, and supports batch-resuming cursors for large list imports.

Category 4: Bookings & Call Log Outcomes

10. CalendarBookings.js

- **What it does:** Detects discovery call bookings and sends pre-call briefing digests.
- **How it works:**
 - Checks your calendar for events starting with the word "Discovery Call" that have external guests.
 - It updates the contact's state to `booked`.
 - Every 15 minutes, it scans for upcoming calls starting in the next 60–75 minutes (or immediately if booked last-minute) and emails Rachel a styled HTML briefing containing the attendee's name, company, tags, workshop attendance, and last 3 email engagements.
 - It includes two custom-encoded mailto links: **WON** and **LOST** (Scenario 16).
 - If a booking is rescheduled, it detects the change and clears the "sent" status to allow re-briefing (Scenario 10). If the event is canceled or declined, it automatically resets the contact's state back to `active` (Scenario 30).

11. OutcomeParser.js

- **What it does:** Automatically logs the win/loss outcome of Rachel's discovery calls.
- **How it works:** Every 15 minutes, it scans Rachel's inbox for unread email threads with subject lines starting with `WON:` or `LOST:`. It extracts the contact's email hash and event ID from the subject line, transitions their lifecycle state accordingly in the sheet, updates EmailOctopus (to unsubscribe won clients from cold nurture lists), and marks the Gmail thread as read. It is fully idempotent (clicking WON twice changes nothing - Scenario 17).

Category 5: Email Campaigns & Reports

12. Newsletter.js

- **What it does:** Automatically creates and schedules the weekly blog newsletter.
- **How it works:** Runs every Thursday at 9:00 AM. It fetches your blog's RSS feed, looks at the newest item, and checks if it was published in the last 7 days and not in the future (Scenario 11). It prevents double-sends by logging sent posts by GUID, compiles the

post into your brand's HTML email template, finds all active/verified subscribers (alerting Rachel if 0 are found - Scenario 32), updates their custom fields in EmailOctopus, and temporarily tags them with `send:newsletter` to fire the EmailOctopus broadcast automation.

13. WebhookReceiver.js

- **What it does:** Listens to EmailOctopus engagement events (opens, clicks, bounces, unsubscribes) in real-time.
- **How it works:** Exposes a public API endpoint (`doPost`). It verifies incoming HMAC-SHA256 signatures to ignore fake traffic (Scenario 12) and checks event IDs against a cache to prevent duplicates.
 - If a subscriber opens or clicks, it updates their `last_engagement_at` timestamp.
 - If a subscriber bounces, it marks them `invalid` in the sheet, marks their status `unsubscribed` in EmailOctopus, and tags them as `flag:invalid-undeliverable`.
 - If a subscriber unsubscribes, it marks them `archived`.
 - If an event arrives before the contact is written to the sheet, it creates a minimal contact instead of throwing an error (Scenario 34).

14. Reports.js

- **What it does:** Emails Rachel a weekly status report and runs system heartbeat checks.
- **How it works:** Runs every Monday at 8:00 AM. It counts total subscribers, weekly growth, validation coverage, duplicate rates, and fetches open/click stats from EmailOctopus. It guards against division-by-zero errors if there were no campaigns (Scenario 35) and incorporates the totals of new and unresolved system errors (Scenario 36). It checks the last runtime of all schedules; if a background job has missed its scheduled time (such as Google silently turning off a trigger), it highlights a loud warning at the top of the email (Scenario 38).

15. Maintenance.js

- **What it does:** Cleans out unresponsive, cold contacts once a month.
- **How it works:** Runs on the 1st of every month at 2:00 AM. It identifies contacts created over 60 days ago who have been emailed but have shown zero engagement. It tags them `stale-60`, runs a re-verification check via ZeroBounce, and archives them if invalid. It features a **circuit breaker** (Scenario 37): if the job attempts to archive more than 20% of the active database in a single sweep, it halts execution immediately, changes nothing, and sends an alert to Rachel (preventing mass data deletions caused by tracking or webhook outages).

Category 6: Entry Points & Testing

16. Triggers.js

- **What it does:** Exposes the main trigger functions that Google's time-based and event-based schedulers bind to.
- **How it works:** It acts as the routing junction, forwarding calendar changes to `CalendarBookings`, spreadsheet submissions to `GoogleForms`, and periodic schedules to their respective managers. It also implements the `doGet(e)` web receiver that processes Rachel's manual "one-click send approval" clicks for newsletters.

17. TestRunner.js

- **What it does:** Executes the 38 stress-test scenarios in-memory to ensure safety before going live.
- **How it works:** It completely overrides the service wrappers in `Services.js` with mock databases, mock calendar arrays, mock email catchers, and mock fetch returns. Running `runTests()` executes assertions verifying that the agent handles race conditions, formats titles correctly, handles outages, handles reschedules, and respects circuit-breaker limits. It writes output logs showing test compliance.

- `Config.js`: Stores all system settings (like account keys and blog links) in one place.
- `Services.js`: Creates a "pretend" environment during testing so the script doesn't send real emails or edit real data.
- `ErrorHandler.js`: Retries tasks if the network drops, and alerts Rachel if a major system failure occurs.
- `Schema.js`: Cleans spelling, fixes capitalization in names, and reformats phone numbers before they are saved.

- Database.js: Handles the physical task of reading from and writing rows to your Google Spreadsheet.
- DedupeMerge.js: Combines new signup details into existing profiles so you don't get duplicate rows for the same person.
- GoogleForms.js: Instantly logs contact info whenever a visitor fills out your blog's subscription form.
- LumaPoll.js: Check your Google Drive once every 15 minutes to import workshop attendee lists.
- LinkedInImport.js: Imports lists of LinkedIn connections, cleaning up messy characters like emojis.
- CalendarBookings.js: Finds client bookings on your calendar and emails Rachel a cheat-sheet about the contact right before the call.
- OutcomeParser.js: Changes a client's status to "Won" or "Lost" when Rachel clicks the links inside her briefing emails.
- Newsletter.js: Builds and schedules the weekly Thursday email campaign when a new post is published on your blog.
- WebhookReceiver.js: Tracks who opened or clicked on your emails, and flags bad emails that bounced.
- Reports.js: Sends Rachel a status report every Monday morning showing new signups, open rates, and system stability.
- Maintenance.js: Quietly archives cold contacts once a month if they haven't opened an email in 60 days.
- Triggers.js: Acts as the master timer, telling the script when to run the weekly newsletters and daily checks.
- TestRunner.js: Runs test drills on the code to make sure the agent doesn't delete database rows or make mistakes under pressure.